24. Използване на XML за структуриране, валидация, обработка и представяне на документно съдържание.
Изложението по въпроса трябва да включва следните по-съществени елементи:
1. Добре-структуриран XML – основни концепции, XML йерархии, синтактични правила. XML пространства от имена.
2. XML валидация чрез Document Type Definitions (DTD) – цели на валидирането, DTD структура, синтаксис.
3. XML валидация чрез XML Schema – спецификации, типове данни, фасети, структури. Сравнение с DTD.
4. Използване на XSLT (eXtensible StyleSheet Language Transformations) и XPath за алокиране, манипулиране и представяне на XML съдържание.
5. Използване на DOM (Document Object Model) и SAX (Simple API for XML) за обработка на XML документи – основни интерфейси на DOM и SAX и начини за използването им. Сравнение между DOM и SAX.

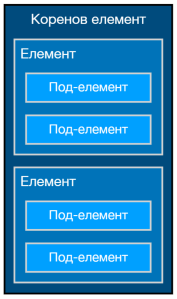
1. **Добре-структуриран XML – основни концепции, XML йерархии, синтактични правила. XML пространства от имена.**

XML означава e**Xtensible Markup Language** и представлява език за маркиране, използван за създаване на **структурирани документи, които могат да бъдат четени както от машини, така и от хора**. Всеки XML документ е изграден от следните компоненти:

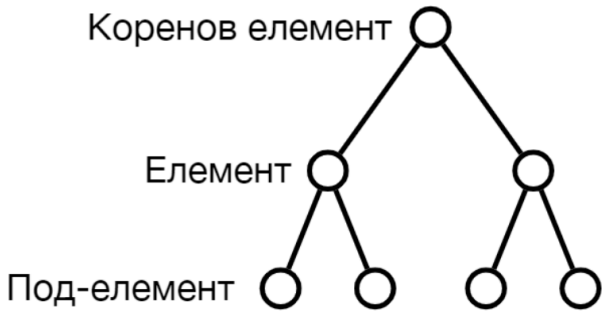
- **Таг** - име на елемент, заградено съм символите “<” и “>” от двете страни. Например <parent>
  - **Отварящ таг** (<tag>) - започва дефиницията на елемент
  - **Затварящ таг** (</tag>) - завършва дефиницията на елемент
- **Елемент** - отварящ и затварящ таг, като между тях има някакво съдържание (текстово или други XML документи). Елементите могат да бъдат влагани като единствено един елемент може да бъде **корен**.
- **Атрибут** - допълнителна информация за елемент. Задава се като **наредена двойка име-стойност**. Стойността на атрибута се огражда от единични или двойни кавички.

```
<person gender=”female”>
        <firstName>Боян</firstName>
        <lastName>Бончев</lastName>
</person>
```

Елементната структура на един XML документ представлява **йерархия**. Може да се представи по два начина: **планирано, дървовидно**:
Пример планирано:



Пример дървовидно:



В една XML йерархия **всеки отварящ таг трябва да им съответстващ затварящ таг.** Освен това, **два или повече елемента не могат да се припокриват** - тоест да започва в един елемент и да завършва след завършването на елемента, в който е започнал (аналогично на балансирането на скоби). Пример: <a> - това не е позволено.

Елементите имат следните синтактични правила:

- ***Имената в XML правят разлика между големи и малки букви. Тоест <person> и <Person> са два различни елемента.***
- ***Имената трябва да започват с главна или малка буква или "_" и могат да бъдат използвани букви, цифри, "_", "-", и "."***
- ***Имената не могат да започват с думите XML, xML, xml, xmL и т.н.***
- ***Празни пространства се допуска само след имената на елементите, т.е преди знака ">"***
- ***Използването на символите "<", ">" е резервирано за разделители***
- ***Символите "&" и ";" се използват за реферирание на съдържание към предефинирана единица (entity)***

Атрибутите имат следните синтактични правила:

- ***Всеки атрибут трябва да има стойност, дори и тя да бъде празен стринг.***
- ***За разделител между два и повече атрибута се използва празно пространство.***
- ***Възможно е едновременно използване на кавички и апострофи за ограждане на стойностите на атрибутите на един елемент. Не е допустимо обаче, ограждането на стойност на един атрибут с кавички и апостроф или обратното.***
- ***Дублиращи се атрибути в един и същ елемент не са разрешени.***
- ***Атрибутите могат да следват произволен ред на подреждане в елемента.***

XML пространства от имена - използва се за **разграничаване на елементи и атрибути**, които могат да имат еднакви имена, но различно значение. Синтаксис: Пространства от имена се дефинират с атрибута ***xmlns****: <book xmlns="http://example.com/books">.*

2. **XML валидация чрез Document Type Definitions (DTD) – цели на валидирането, DTD структура, синтаксис**.

Цели на валидирането

- **Верификация на структурата, проверка за коректност** - Уверява, че XML документите следват предварително дефинирана структура и правила.
- **Съответствие**: Гарантира, че документът е съвместим с определени стандарти; гарантира се, че XML документът има определена структура, която се очаква от приложението, което ще го обработва

DTD структура - DTD задава правила, съгласно които се определят **имената на елементите и атрибутите, тяхната последователност, честота на срещане** и т.н.
Вътрешен (inline) DTD: Включен в самия XML документ.
Външен DTD: Съдържа се в отделен файл и се реферира в XML документа.
Като в един документ можем да имаме едновременно както вътрешно DTD, така и външно DTD.

- **Деклариране на елементи**: *<!ELEMENT element_name (content_model)>*
    - Съдържанието може да бъде:
        - **Елементно** - тоест съдържа други елементи, които трябва да имат дефиниция в DTD. Например: *<!ELEMENT person (father, mother, children)>;*
        - **Смесено** - в тази декларация ключовият елемент **#PCDATA** трябва да предхожда всички други елементи. Различните възможност за съдържание се отделят с "**|**". Например: *<!ELEMENT person (#PCDATA | a | b)>*
        - **Празно** - указва се с ключовата дума **EMPTY**
        - **Произволно** - оказва се с ключовата дума **ANY**
    - В дефиницията за съдържание може да използваме и индикатори за брой срещания
        - **+** - даденият елемент трябва да се съдържа поне един път
        - ***** - даденият елемент може да се 0 или повече пъти
        - **?** - даденият елемент трябва да се съдържа тончо 1 път или нула пъти
- **Деклариране на атрибути:**
    - Атрибутите могат да бъдат прикрепени към декларацията на елементите или да бъдат в отделна декларация;
    - Декларацията се състои от три части:
        - Ключовата дума **ATTLIST**;
        - **Името на елемента**, на когото тези атрибути принадлежат;
        - **Списък с декларациите на атрибутит**е – за всеки атрибут се дават **име, тип и декларация на стойността**.
    - Декларацията на стойността може да бъде – стойност по подразбиране, фиксирана стойност, изискваща се (**#REQUIRED**) или опционална (**#IMPLIED**)
    - Някои възможни типове на атрибутите:
        - **CDATA** – указва, че атрибутът има стойност от символи (Character Data), които няма да се интерпретират от парсера;

        - **ID** – указва, че стойността на атрибута служи като ключ на елемента и той може да се идентифицира еднозначно чрез нея;
        - **IDREFS** – стойността на атрибута е списък от **IDREF** стойности, които са разделени с празно място;
        - **ENTITY** – стойността на атрибута е референция към външна единица, съдържаща не-XML съдържание;
        - **ENTITIES** – списък от **ENTITY** стойности, разделени с празно място
    - Пример

```
<!ATTLIST book
        id ID #REQUIRED
        genre CDATA #IMPLIED
>
```

- **Декларация на единици: <!ENTITY …>** - използва се като указател към текст или към външен ресурс
- **Декларация на нотация: <!NOTATION …>** - използва се за описване на данни, които не са XML, като например: снимки, анимации и др.
- Пример за DTD схема:

```
<?xml version="1.0"?>
<DOCTYPE root-name [
        <!ELEMENT newspaper (article+)>
        <!ELEMENT article (heading, body, notes)>
        <!ELEMENT heading (#PCDATA)>
        <!ELEMENT body (#PCDATA)>
        <!ELEMENT notes (#PCDATA)>

        <!ATTLIST article author CDATA #REQUIRED>
        <!ATTLIST article editor CDATA #IMPLIED>
        <!ATTLIST article date CDATA #IMPLIED>

        <!ENTITY  newspaper "Financial times">
        <!ENTITY publisher "Sofia press">

        <!NOTATION jpeg PUBLIC "image/jpeg">
        <!ENTITY logo SYSTEM "logo.jpg" NDATA jpeg>
]>
```

3. **XML валидация чрез XML Schema – спецификации, типове данни, фасети, структури. Сравнение с DTD**.

XML schema е XML документ, който **описва структурата на друг XML документ**. Тя е наследник на DTD, но предоставя много по-богати възможности за по-прецизна спецификация на XML структура.

<xs:schema> – коренът на всеки схема документ.
• <xs:element> – използва се за деклариране на елементи
• <xs:attribute> – използва се за декларация на атрибути
• <xs:simpleType> – дефинира елемент от прост тип.
• <xs:complexType> – дефинира елемент от сложен тип

• <xs:attributeGroup> – позволява декларацията на група от атрибути
• <xs:group> – позволява да се групират елементи и така получената група да се преизползва на различни места. В една група могат да присъстват единствено елементи. Редът на елементите в групата се задава с помощта на долните три декларации.

○ <xs:all> – съдържа декларации на елементи, които трябва да присъстват в XML документа, независимо в какъв ред;

○ <xs:choice> – съдържа декларации на елементи, от които точно един може да присъства в XML документа;

○ <xs:sequence> – съдържа декларации на елементи, които трябва да присъстват в XML документа в точно указания ред (последователно).

Типове данни

- **Прости типове:** integer, boolean, float, double, ID, date и др.
- **Сложни типове:** използвани са други прости и сложни типове, за да се дефинират. Те се изграждат на базата на: <xs:simpleType>, <xs:complexType>

Фасети - Определят **допустимите стойности** за елементите и атрибутите, като например дължина, стойностен диапазон и формат.
Съществуват два вида фасети:

- **Фундаментални фасети** като: *Equal, Order, Bounds, Cardinality, Numeric*
- **Ограничаващи фасети** като: minInclusive/minExclusive; length(определя броя на елементите/символите в даден елемент); minLength/maxLength; pattern(ограничава низов тип с регулярен израз); enumeration (изброяване на допустими стойности); precision (броя на цифрите от дясно на десетичната запетая); scale (броя на цифрите от ляво на десетичната запетая)

Структури:

- дефиниция на типове: <xs:simpleType>, <xs:complexType>
- дефиниция на атрибути: <xs:attribute>
- дефиниция на елементи: <xs:element>
- дефиниция на групи: <xs:group>
- дефиниция на групи от атрибути: <xs:attributeGroup>
- декларация на нотации: <xs:notation>

Сравнение с DTD

- XML Schema е по-интуитивен от DTD, понеже е по-близък до XML синтаксиса
- може да се задават много по-стриктни правила с XML Schema. XML Schema е много по-богат и мощен
- DTD е по-прост и лесен, защото използва по-стегнат и кратък синтаксис

4. **Използване на XSLT (eXtensible StyleSheet Language Transformations) и XPath за алокиране, манипулиране и представяне на XML съдържание**

XSLT е език за **трансформация** на XML документи в други формати, като HTML, текст или друг XML. XSLT позволява на потребителите да **дефинират шаблони и правила за трансформацията**, базирани на структурата и съдържанието на оригиналния XML документ.

В XSLT има дефинирани 34 елемента като: <xsl:stylesheet>, <xsl:template>, <xsl:apply-templates>, <xsl:value-of>, <xsl:for-each>, <xsl:output>,, <xsl:number>, <xsl:variable>, <xsl:element>, <xsl:attribute>, <xsl:copy>, <xsl:copy-of> и др. Като например с помощта на <xsl:variable> можем да дефинираме променлива, а с помощта на <xsl:element> може да създадем нов елемент и позволява динамично създаване на елементи.

В един XML документ XSLT се задава посредством инструкцията:
<?xml-stylesheet type="text/xsl" href="stylesheet.xsl">

XPath - **език за навигация и селекция** на части от XML документите. В XPath повечето съставни части на документа се представят като възли на дърво. Коренът на XPath дървото представлява самия документ и се бележи "/"

Един XPath израз е текстов низ чрез, който се избират един или няколко възли в дървото.
Видове XPath възли:

- **корен** - сочи към корена на документа
- **елемент** - представя елемент
- **атрибут** - представя атрибут
- **текст** - представя текстово съдържание на елемент
- **инструкция за обработка**
- **декларация на CDATA** секция (област от данни)
- **коментар**

В XPath съществуват два вида **пътища**: **относителни**(започват от **контекстния възел** - могат да заповат с низа "./") и **абсолютни**(започват от **корена на дървото** - започват с низа "/").
Използвайки XPath можем да извършим следните действия:

- да селектираме element, който има атрибут *id* с определена стойност: *(library/book[@id='my_book'])*
- да селектираме всички елементи от даден тип: *library/book*
- след като селектираме елемент можем да вземем:
    - всички негови преки наследници
    - родителя му
    - атрибутите му
    - възлите, които са след него
    - и др.

XPath използва синтаксис подобен на адресирането на директории:

- ".." за адресиране на родителя на контекстния възел
- "." за адресиране на текущия възел
- "*" за избор на всичко
- "@" за атрибут. Например селектиране на елементи, които имат даден атрибут равен на някаква стойност: library/book[@genre="fiction"]

5. Използване на DOM (Document Object Model) и SAX (Simple API for XML) за обработка на XML документи – основни интерфейси на DOM и SAX и начини за използването им. Сравнение между DOM и SAX.

DOM представя XML документите като дървовидна структура, която може да бъде манипулирана чрез програмни интерфейси. Позволява ни лесно да навигираме в документа и да го променяме, но консумира повече памет, особено при по-големи документи. DOM е част от съвременните браузъри и е в основата на динамичните уеб страници, защото чрез DOM също може динамично да се променят, премахват и добавят елементи в една HTML страница.
Базови интерфейси:

- Node - представлява един връх в документното дърво
- Document - представлява целия XML документ (корена на дървото)
- Element - представлява един елемент в даден XML документ. Докато Node може да бъде елемент, текстово съдържание, атрибут и т.н, Element може да бъде само XML елемент.
- Attr - свойства name и value, които връщат името и стойността на атрибута
- CharacterData - съфържа полезни методи за текстообработка
- Text - тези възли винаги за листа
- Comment - представлява коментар
- NodeList - колекция от наредени Node обекти
- NamedNodeMap
- DocumentFragment

Разширени интерфейси

- CDATASection - представя CDATA секция
- DocumentType - информация за съдържанието на DTD
- ProcessingIntruction
- EntityReference
- Entity
- Notation

SAX (Simple API for XML) - Обработва XML документи на базата на събития. Разработен е за по-ефективен анализ на големи XML документи (не се налага всеки път да прочита целите документи). Консумира по-малко памет.
Основни SAX интерфейси:

- DocumentHandler
- ContentHandler
- XMLReader
- XMLFilterImplt
- XMLReaderAdapter
- ErrorHandler - три нива на обработка на грешки: warning, error, fatalError
- DTDHandler
- EntityResolver

Сравнение между DOM и SAX

- SAX е подходящ за четене на големи XML документи и обработка в реално време
- DOM разглежда XML като логическо дърво, докато SAX е базиран на събития
- DOM е добър при обработка на много елементи и структурирани промени на документа